

Jour 7 : Le Mode Utilisateur (Ring 3)

1. La Protection Matérielle : Les "Rings"

L'architecture x86 offre 4 niveaux de privilèges, appelés **Rings** (Anneaux), numérotés de 0 à 3.

- **Ring 0 (Kernel Mode)** : Le code a accès à toutes les instructions (y compris les dangereuses comme cli) et à toute la mémoire. C'est là que vit notre noyau actuel.
- **Ring 3 (User Mode)** : C'est là que tourneront les applications (calculatrice, shell, navigateur...).
 - Interdiction d'exécuter des instructions privilégiées.
 - Interdiction d'accéder à la mémoire du Ring 0 (protection via la Segmentation et/ou la Pagination).

Note : Les Rings 1 et 2 sont rarement utilisés par les OS modernes.

Pourquoi séparer ? (Défense en profondeur)

Si tout tourne en Ring 0 (comme c'est le cas dans notre OS actuellement), un simple bug dans une application utilisateur (ex: un pointeur fou) peut écraser la mémoire du noyau et faire planter tout le PC. En Ring 3, si une application fait une bêtise, le CPU déclenche une exception. Le noyau (Ring 0) attrape cette exception et peut fermer *juste* l'application fautive sans crasher le système entier.

2. Rôle de la GDT dans la protection

La GDT (Global Descriptor Table) définit les segments de mémoire. Chaque descripteur de segment possède un champ **DPL (Descriptor Privilege Level)** sur 2 bits.

- Jusqu'à présent, nous avons :
 - Code Segment (Kernel) : DPL = 0
 - Data Segment (Kernel) : DPL = 0
- Pour le User Mode, nous devons ajouter :
 - Code Segment (User) : DPL = 3
 - Data Segment (User) : DPL = 3

Le CPU vérifie en permanence le **CPL (Current Privilege Level)**, qui est stocké dans les 2 derniers bits du registre CS (Code Segment).

- Si CS termine par 00 (binaire) -> On est en Ring 0.
- Si CS termine par 11 (binaire) -> On est en Ring 3.

3. Le TSS (Task State Segment) - Le point subtil

C'est souvent l'oubli fatal qui cause des "Triple Faults". Quand on est en Ring 3 et qu'une interruption survient (ex: le Timer, le Clavier, ou notre fameux General Protection Fault), le CPU doit repasser en Ring 0 pour exécuter le handler de l'interruption.

Problème : En Ring 3, on utilise une stack "utilisateur". Le CPU ne peut pas utiliser cette stack pour y pousser les registres du noyau (sécurité). Il a besoin de savoir **où se trouve la stack du noyau (Ring 0)** pour basculer dessus.

C'est le rôle du **TSS**. Même si on ne fait pas de multitâche matériel complexe, on a besoin d'un TSS minimal qui contient une seule information cruciale : **ESP0** (Le pointeur de stack pour le Ring 0). Sans TSS configuré, dès qu'une exception arrivera en Ring 3, le CPU ne saura pas où aller, et c'est le crash (Triple Fault).

4. Le Grand Saut : iret (Interrupt Return)

Comment passer de Ring 0 à Ring 3 ? On ne peut pas faire un simple jmp. On utilise une astuce. L'instruction iret sert normalement à revenir d'une interruption (donc revenir de Ring 0 vers ce qu'on faisait avant). L'astuce consiste à **feinter le CPU** :

1. On pousse sur la stack ("la main") les valeurs que le CPU *aurait* sauvées s'il y avait eu une interruption venant du Ring 3 :
 - **SS** (User Data Segment)
 - **ESP** (Pointeur stack utilisateur)
 - **EFLAGS** (Drapeaux, attention à réactiver les interruptions ici !)
 - **CS** (User Code Segment)
 - **EIP** (L'adresse de la fonction utilisateur)
2. On exécute iret.
3. Le CPU "restaure" ces registres. En chargeant CS avec notre sélecteur User (qui a les bits 11 à la fin), le CPU passe instantanément en Ring 3 !

```
// Déclaré dans process.s
extern void switch_to_user_mode(void (*f)());

void user_main() {
    fb_print("--- USER MODE (RING 3) ---\n", 13); // Magenta
    fb_print("Hello from User Mode! I have limited power here.\n", 15);

    // Test 1: Calcul simple (Autorisé)
    int x = 10;
    int y = 20;
    int z = x + y;
    if(z == 30) {
        fb_print("Calcul User OK.\n", 10);
    }

    // Test 2: Tenter une instruction interdite (CLI)
    fb_print("Tentative d'exécution instruction privilegiee (CLI)...\n", 14); // Jaune
    fb_print("Si tout va bien, le Noyau va m'arreter via GPF...\n", 14);

    // Cette instruction doit déclencher une General Protection Fault (#GPF)
    asm volatile("cli");

    fb_print("FATAL: J'ai survécu au CLI ! La protection a echoue.\n", 4);
    while(1);
}
```

et on fait le handler d'interruption dans les fichier idt.c et interrupts.s

```
GNU nano 7.2 idt.c *
}

void interrupt_handler_0() {
    fb_print("!!! EXCEPTION 0: DIVISION PAR ZERO !!!\n", 4);
    while(1);
}

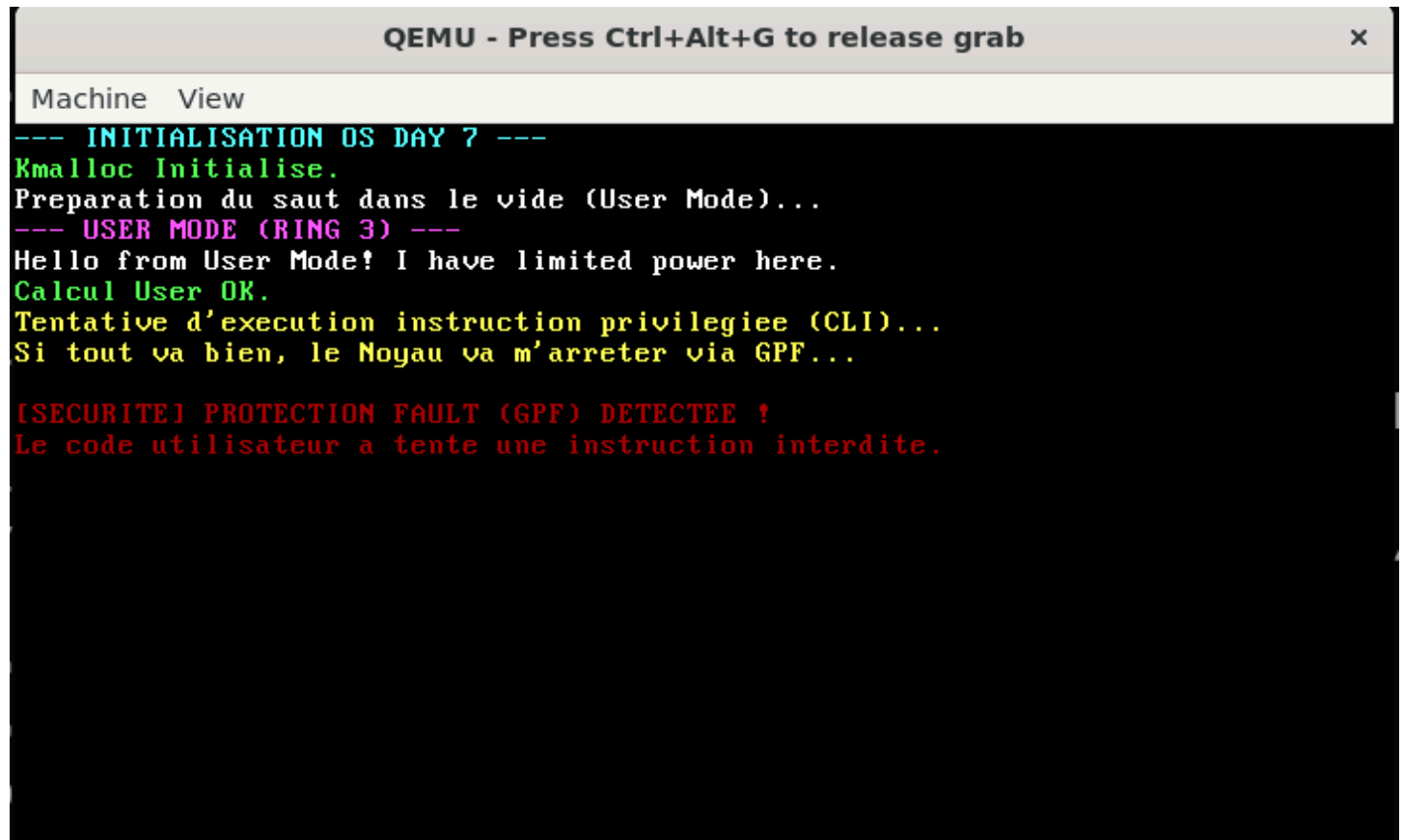
void interrupt_handler_13() {
    // Ne pas effacer l'écran pour voir l'historique
    fb_print("\n[SECURITE] PROTECTION FAULT (GPF) DETECTEE !\n", 4); // Rouge
    fb_print("Le code utilisateur a tente une instruction interdite.\n", 4);
    fb_print("-> Processus termine par le noyau.\n", 14); // Jaune

    // Le processeur est maintenant en sécurité (Ring 0)
    // on simule la fin du processus par une boucle infinie silencieuse (ou HLT)
    while(1) {
        asm volatile("hlt");
    }
}

void interrupt_handler_14() {
    fb_print("\n>>> PAGE FAULT DETECTE ! (Acces memoire interdite) <<<", 0x4F); // Rouge/Blanc
    while(1);
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo

Cette interruption est essentielle sinon la vm va redémarrer à chaque fois car elle sera face à une "Protection Fault".



```
QEMU - Press Ctrl+Alt+G to release grab
Machine View
--- INITIALISATION OS DAY 7 ---
Kmalloc Initialise.
Preparation du saut dans le vide (User Mode)...
--- USER MODE (RING 3) ---
Hello from User Mode! I have limited power here.
Calcul User OK.
Tentative d'execution instruction privilegiee (CLI)...
Si tout va bien, le Noyau va m'arreter via GPF...

[SECURITE] PROTECTION FAULT (GPF) DETECTEE !
Le code utilisateur a tente une instruction interdite.
```

et comme vous le voyez, la ligne ci-dessous n'a pas été exécutée.

```
fb_print("FATAL: J'ai survécu au CLI ! La protection a echoue.\n", 4);
```